

Autor: Joosep Tiger Tilgar (individuaaltöö)

8. Rühm, grupp H

Toidumajanduse andmebaas

Etapp 1

Valdkonna kirjeldus ja probleemi sõnastus

Toiduülejääkide majandamise valdkond hõlmab toidu annetajaid, annetuste vastuvõtjaid ja mis kõige tähtsam - täpset logistikat, et hõlbustada väärtusliku söögikõlbliku toidu haldust, edasi jagamist ning takistada toidukauba asjatut raisku minemist. Toiduraiskamine on ülemaailmne probleem, mis on tihtipeale põhjustatud tänapäevasest mugavusühiskonnast, kus toidukaup on lihtsasti ja laialdaselt kättesaadav ning sellest tulenevalt ei suudeta kogu toorainet plaanipäraselt ära tarbida. Selle probleemi lahenduseks on loodud toiduannetussüsteemid, mis ühendavad toidutootjad, poed, restoranid, eraisikud, toidupangad ja muud heategevusorganisatsioonid ning pakuvad neile logistilist võrgustikku, mis võimaldab igal osapoolel teineteisega ühendust võtta ja toidukaupa vahetada.

Mõisted

- **Annetus** - toidukohtadest või muudest asutustest üle jäänud toidu andam, mille säilivusaeg on lõppenud või peagi lõppemas ja mida pole ärilistel põhjustel lubatud enam ressursina kasutada, ning mida on tervikuna võimalik anda edasi kellelegi, kes saaks sellest kasu.
- **Annetaja** - toidupood, tooja või eraisik, kes annetab liigset ja kasutuskõlblikku toitu.
- **Vastuvõtja** - heategevusorganisatsioon, toidupank või eraisik, kes võtab annetuse vastu ja vajadusel jagab seda edasi sinna kuhu vaja.
- **Transporditeenus** - transporditeenuse pakkujad, kes toimetavad toidu annetajalt vastuvõtjale ning hoolitsevad süsteemi logistilise osa toimimise eest.
- **Toidukategooria** - vahendatava toiduaine tüüp (näiteks piimatooted, puuviljad, pagaritooted jne).

Valdkonna probleemid

- 1) **Toidu raiskamine** - suurtes kogustes tarbimiskõlblikku toitu visatakse ära, kuigi see võiks aidata paljusid abivajajaid.
- 2) **Halb logistika** - tihtipeale puudub efektiivne süsteem, mis jälgiks annetuste ja transpordi liikumist ning annaks ülevaate saadavalolevatest ülejääkidest.
- 3) **Ühenduste puudumine** - annetajad ja vastuvõtjad ei pruugi üksteist leida või ei tea, milla ja kui palju toitu saadaval on.
- 4) **Statistilised raskused** - paljude osapoolte tõttu on raske toiduülejääkide majandamise kohta statistikat teha.

Probleemid, mida andmebaas lahendab

- 1) Võimaldab annetusi jälgida: andmebaasis salvestatakse kes annetab mida ja millises koguses. Andmebaas vähendab toidu raiskuminekut aidates annetajatel lihtsamini leida vastuvõtjaid ja abivajajaid, mistõttu jõuab info kasutuskõlblikust toiduülejäägist rohkemate inimesteni, suurendades võimalust, et ülejäänud toit ei lähe raisku ja leiab koha, kus seda vajatakse.
- 2) Saab hõlpsasti jälgida, kuidas annetused liiguvad annetajalt vastuvõtjani ja hallata logistilisi protsesse, mis kõik aitab vähendada potentsiaalseid logistikaprobleeme.
- 3) Süsteemile saab rakendada automaatset sobitamist, mis saab aidata leida aktiivsele annetusele lähima ja sobivaima vastuvõtja.
- 4) Andmebaasist saab kiiresti väljastada annetuste kogused, populaarsemad toidukategooriad ja üleliigse toidu tekkimise trende ning on lihtsam teha statistilisi analüüse ja aruandeid süsteemi toimimise efektiivsuse või logistika kohta.

Kasutajate ja nende vajaduste kirjeldus

- 1) **Annetajad** (poed, restoranid, tootjad, eraisikud):
 - Milliseid toiduaineid vajatakse kõige rohkem?
 - Kas minu annetus on juba vastu võetud?
 - Kuidas saan ülejääke kõige efektiivsemalt ära anda?
- 2) **Vastuvõtjad** (toidupangad, varjupaigad, abiorganisatsioonid):

- Milliseid annetusi on saadaval?
- Millal jõuab toiduabi kohale?
- Kellele peaksin / saaksin toidu jaotada?
- Kui pikk on toiduaine eeldatav säilivusaeg?

3) **Transporditeenuse pakkujad** (kullerifirmad, vabatahtlikud):

- Millised annetused vajavad transporti?
- Kus saadetised hetkel asuvad, kas on tekkinud mingeid tõrkeid?

4) **Administraatorid ja analüütikud:**

- Missugused kasutajad on osa võrgustikust?
- Kui palju toitu on annetatud?
- Millised on kõige aktiivsemad annetajad ja vastuvõtjad ning kõige efektiivsemad transportijad?
- Kus asuvad logistikaketi kitsaskohad?
- Millised on ülejääkide majandamise trendid?

Ärireeglitest tulenevad seosed ja seoste aarsused

1) **Annetaja - Annetus (1:M): (andmeliisasuse tõttu eemaldatud)**

- Iga annetus on suunatud ühele annetajale.

2) **Vastuvõtja - Annetus (1:M): (andmeliisasuse tõttu eemaldatud)**

- Ühte annetust võib vastu võtta ainult üks vastuvõtja.

3) **Annetus - Annetuse_toiduained (1:M):**

- Iga annetus võib sisaldada mitut erinevat toiduainet.

4) **Toiduained - Annetuse_toiduained (1:M):**

- Mitu annetuses olevat toiduaine kirjet võivad viidata ühele toiduainetüübile. Üks toiduaine võib esineda korraga mitmes erinevas annetuses ja seega peaks ühe toiduainega olema seotud mitu kirjet.

5) **Kasutaja - Aadress (1:1):**

- Igal kasutajal on unikaalne aadress.

6) **Aadress - Transport_annetus (1:1):**

- Igal Annetusel on kindel alguskoha ja sihtkoha aadress.

7) **Kasutaja - Annetus, Transport, Vastuvõtja (1:M):**

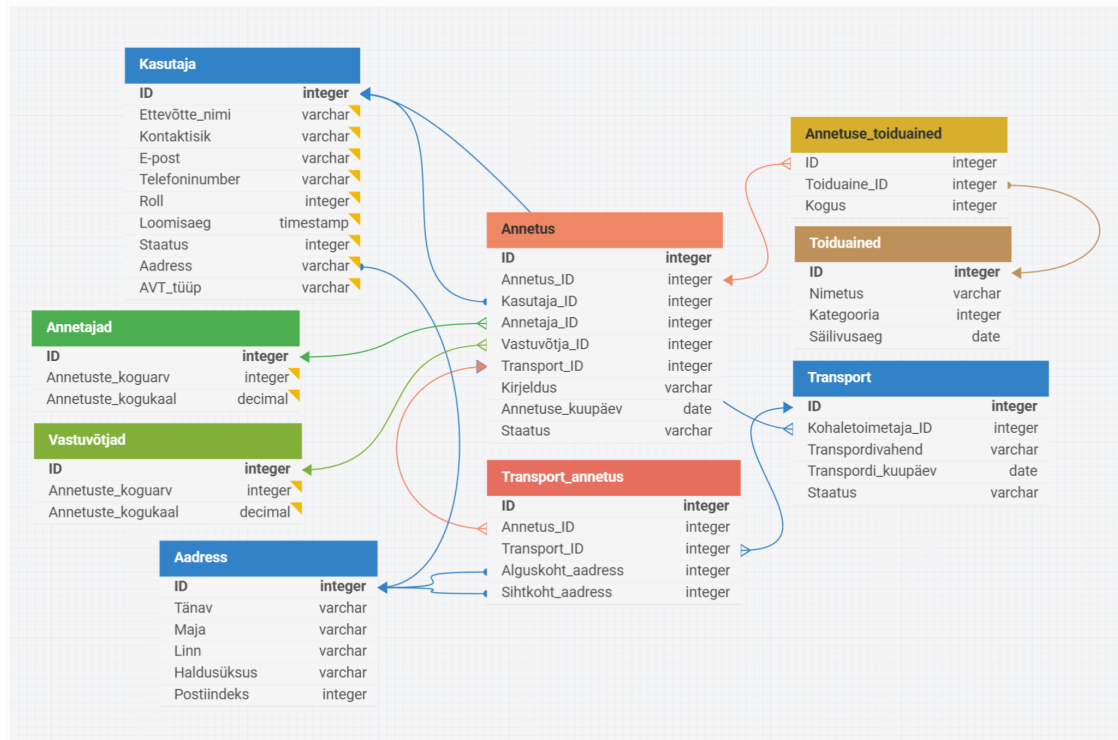
- Andmebaasi kasutajad täidavad erinevaid rolle: annetajad, transportijad ja vastuvõtjad).

8) **Annetus - Transport** vahel on seosetabel **Transport_annetus**, mis loob nende vahel **M:M** seose, kuna üks annetus võib olla jagatud mitme transportsõiduki vahel ja üks transportsõiduk võib sisaldada mitut annetust.

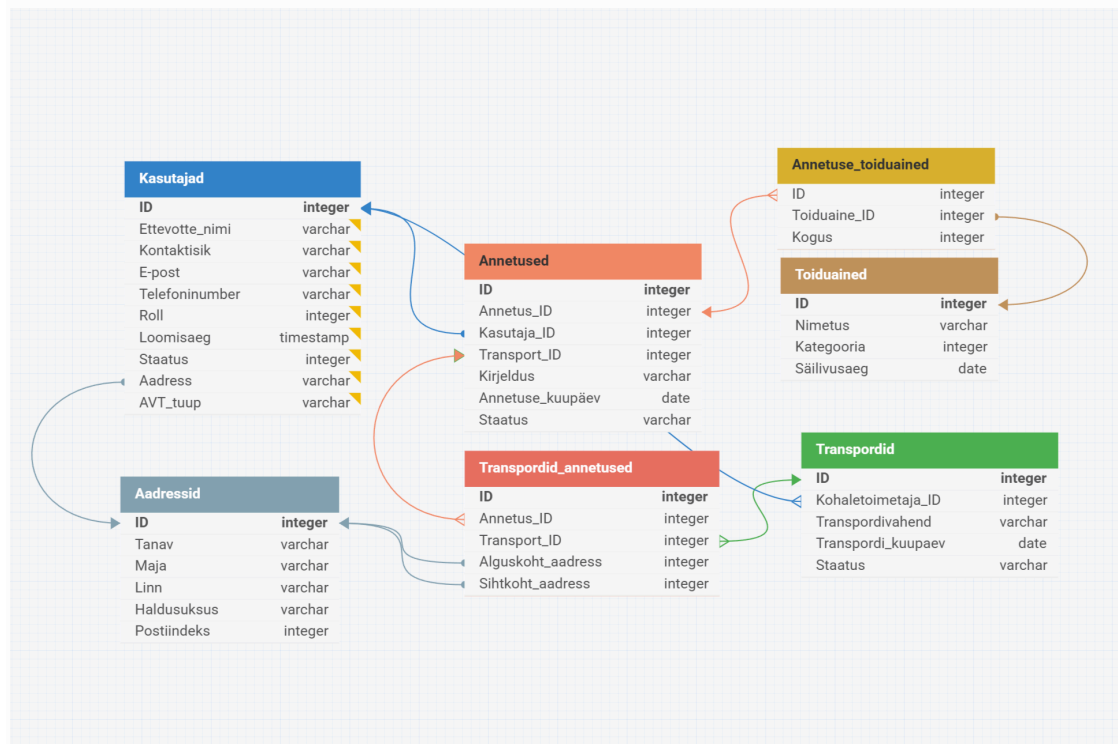
- **Transport (ID) - Transport_annetus (1:M):** Üks transportija võib toimetada mitu annetust korraga. Transport tabel hoiab üldist infot transpordi kohta, aga mitte aadresse.
- **Annetus (ID) - Transport_annetus (1:M):** Üks annetus võib olla jagatud mitme transportsõiduki vahel.
- **Aadress (ID) - Transport_annetus (1:M):** Transport_annetus tabel määrab igale transporditavale annetusele eraldi algus- ja sihtkoha ning sama aadressi saab kasutada mitmes annetuses ja transpordis.

Olem-seose (ER) diagramm

Algne model:



Uuendatud model:



Mudeli selgitus

- 1) **Tabel Kasutaja:** info kasutaja kohta. Iga kasutaja kohta on märgitud ettevõtte, mille nime alt tegutsetakse (juhul kui see eksisteerib ehk võib olla NULL), kontaktisiku nimi, kontaktisiku E-posti aadress, kontaktisiku telefoninumber, kasutaja roll süsteemis (nt administraator, toiduhaldur, vastuvõtja, töötaja, transportija või niisama väline jälgija), kasutajakonto loomisaeg, konto seisund (nt “aktiivne”, “mitteaktiivne” või “määratlemata”) ja aadress. Viimaseks atribuudiks on AVT tüüp, mis määrab, kas kasutaja on hetkel annetaja, vastuvõtja või transportija rollis (väärtus võib olla null).

(andmeliisasuse tõttu eemaldatud)

- 2) **Tabel Annetajad:** sisaldab organisatsioone või asutusi, kes annetusi vastu võtavad ja jaotavad. Iga vastuvõtja on seotud ID kaudu tabeliga Annetus. Põhilised atribuudid on vastuvõtja annetuste koguarv ja nende kogukaal. Mõlemad suurused arvutatakse tehtud tehingute pealt.

(andmeliisasuse tõttu eemaldatud)

- 3) **Tabel Vastuvõtjad:** sisaldab organisatsioone või asutusi, kes annetusi vastu võtavad ja jaotavad. Iga vastuvõtja on seotud ID kaudu Annetus tabeliga. Põhilised atribuudid on vastuvõtja annetuste koguarv ja nende kogukaal. Mõlemad suurused arvutatakse tehtud tehingute pealt. Kõik atribuudid on Annetajate tabeliga identsed.
- 4) **Tabel Annetus:** haldab annetatud toidukoguseid ja nende olekut. Igal annetusel on oma identifikaator number ehk ID, annetaja ID, vastuvõtja ID, toidukategooria ID ja transpordi ID (nullitav). Samuti on olemas annetuse kirjeldus, kogus (kg / tk), eeldatav säilivusaeg (arvutatav), annetuse kuupäev ja hetkestaatus, mis iseloomustab annetuse praegust seisut (nt “ootel”, “saadetud” või “vastu võetud”). Igal annetusel ja transpordil on eraldi staatus ja need pole sünkroonis.
- 5) **Tabel Annetuse_toiduained:** kategoriseerib annetatud toiduaineid, mis on koos ühes annetuses. Atribuutideks on ID, mis seob erinevate toiduainete nimekirja konkreetse annetusega, toiduaine ID (et süsteemis saaks mitu samanimelist toodet eristada, samuti on ID otsing ja muutmine kiirem), kogus ja keskmine säilivusaeg.

- 6) **Tabel Toiduained:** hoiab infot toiduainete kohta. Atribuutideks on ID, toiduaine nimetus, tüüp või kategooria ja säilivusaeg.
- 7) **Tabel Transport:** jälgib annetuste transporti annetajalt vastuvõtjale. Atribuutideks on ID, mis seob selle vahetabeliga Transport_annetus, kohaletoimetaja ID, sõiduki tüübi täpsustus, ja staatuse väli, sest kuigi ühes sõidukis võib olla mitu annetust ja sõiduk võib teha mitmeid peatusi, siis on vaja terve väljasõidu progressi jälgida. Transportija staatus jälgib saadetise olekut (nt “ootel”, “laadimisel”, “teel”, “kohale jõudnud”, “lõpule viidud”, “tühistatud” või “viibib”). Igal annetusel ja transpordil on eraldi staatus ja need pole sünkroonis.
- 8) **Tabel Transport_annetus:** kuna üks transport võib sisaldada mitut annetust ja üks annetus võib olla mitmes transpordis, siis tegemist on vahetabeliga, mis vahendab Annetus ja Transport tabeli M:M seost. Annetuse ID viitab annetuse tabelile ja transportaluse ID viitab omakorda Transport tabelile. Tabel sisaldab ka atribuute alguskoht_aadress ja sihtkoht_aadress, mille kaudu on võimalik pääseda otse ligi vajalikele aadressitele.

Etapp 2

Muudatused ja 1. etapi parandsued

- 1. etapi parandused on märgitud punasega, lisad ja täpsustused aga roheline värviga.
- Said parandatud välja toodud õigekirjavead, mis on praeguseks märgitud lahendatuks (nendele vastavaid kommentaare enam ei eksisteeri).
- Sai lisatud mõiste “annetus” selgitus.
- Täpsustati esimese probleemi lahendust, mida polnud esmapilgul kõigil lugejatel lihtne tabada.

- Kuigi andmebaasi mudel oli juba osaliselt relatsioonilisel kujul ning mitte enam veripuhtal ER-diagrammi kujul, siis pole ID-de ja vahetabelite, nagu Transpot_annetus, tagantjäre eemaldamisel mõtet, sest see oleks mõttetut tööprotsessi taandareng.
- Relatsioonilisest mudelist eemaldati tabelid Annetajad ja Vastuvõtjad, sest neis olevad annetuste koguarvu ja kogukaalu väljad, mis pidid algse plaani järgi olema arvutatavad, on tegelikult võimalik väljastada teiste relatsioonide kaudu, mistõttu on need tabelid koos oma aarsustega andmeliiasuse vältimiseks nüüdseks eemaldatud.
- Samuti on normaliseeritud kõik tabelite ja atribuutide nimetused: tabelite nimed on mitmuses ning ükski tabelinimi ega atribuut ei sisalda täpitähti.
- Vastuseks remargile tabeli "Kasutajad" olemasolu kohta: nimelt on kasutajate tabel loodud selle mõttega, et iga andmebaasi kasutaja kohta käiv info oleks lihtsasti kättesaadav ühest tabelist. Kuna selline andmebaas oleks tõenäoliselt integreeritud mingisugusesse interaktiivsesse veebiraknedusse, kuhu saavad lisaks toiduvõrgustiku osalistele pääseda ligi ka välised vaatajad, kes võiksid potentsiaalselt võrgustikuga liituda, siis on vajalik, et iga osaline saaks luua rakendusse, ja seega ka andmebaasi, oma kasutajakonto.

Relatsioonide funktsionaalsed õltuvused ja normaalkujud

Relatsioonid 3. Normaaltasemlee viiduna:

1. Annetused (**annetus_ID**, kasutaja_ID, transport_ID, kirjeldus, annetuse_kuupaev, staatus)
2. Transpordid (**transport_ID**, kohaletoiimetaja_ID, transpordivahend, transpordi_kuupaev)
3. Kasutajad (**e-post**, kontaktisik, ettevõtte_nimi, **telefoninumber**, roll, loomisaeg, staatus, aadress_ID, AVT_tuup) (võti on kombinatsioon)
4. Aadressid (tanav, maja, linn, **postiindeks**)
5. Postiindeksid (**postiindeks**, haldusüksus)

6. Transpordid_annetused (**annetus_ID**, transport_ID)
7. Transpordi_marsruudid (**transport_ID**, alguskoht_aadress_ID, sihtkoht_aadress_ID)
8. Annetuse_toiduained (**annetus_ID**, **toiduaine_ID**, kogus) (võti on kombinatsioon)
9. Toiduained (**nimetus**, kategooria)
10. Toidukategooriad (**kategooria**, sailivusaeg)

Tehtud dekompositsioonid:

1. Algne:

Aadressid (tanav, maja, linn, haldusüksus, **postiindeks**)

Leidub transitiivne sõltuvus:

(tanav, maja, linn) → postiindeks ja postiindeks → haldusüksus

Dekompositsioon:

Aadressid (tanav, maja, linn, **postiindeks**)

Postiindeksid (**postiindeks**, haldusüksus)

2. Algne:

Transpordid_annetused (**annetus_ID**, **transport_ID**, alguskoht_aadress_ID, sihtkoht_aadress_ID)

Kuna transport_ID on kordumatu, siis kehtib transitiivne sõltuvus: transport_ID → alguskoht, sihtkoht, mispuhul tekib kahe võtme konflikt.

Dekompositsioon:

Transpordid_annetused (**annetus_ID**, transport_ID)

Transpordi_marsruudid (**transport_ID**, alguskoht_aadress_ID, sihtkoht_aadress_ID)

3. Algne:

Toiduained (**nimetus**, kategooria, säilivusaeg)

Leidub transitiivne seos, mida ei tohiks eksisteerida:

nimetus → kategooria, säilivusaeg ja kategooria → säilivusaeg

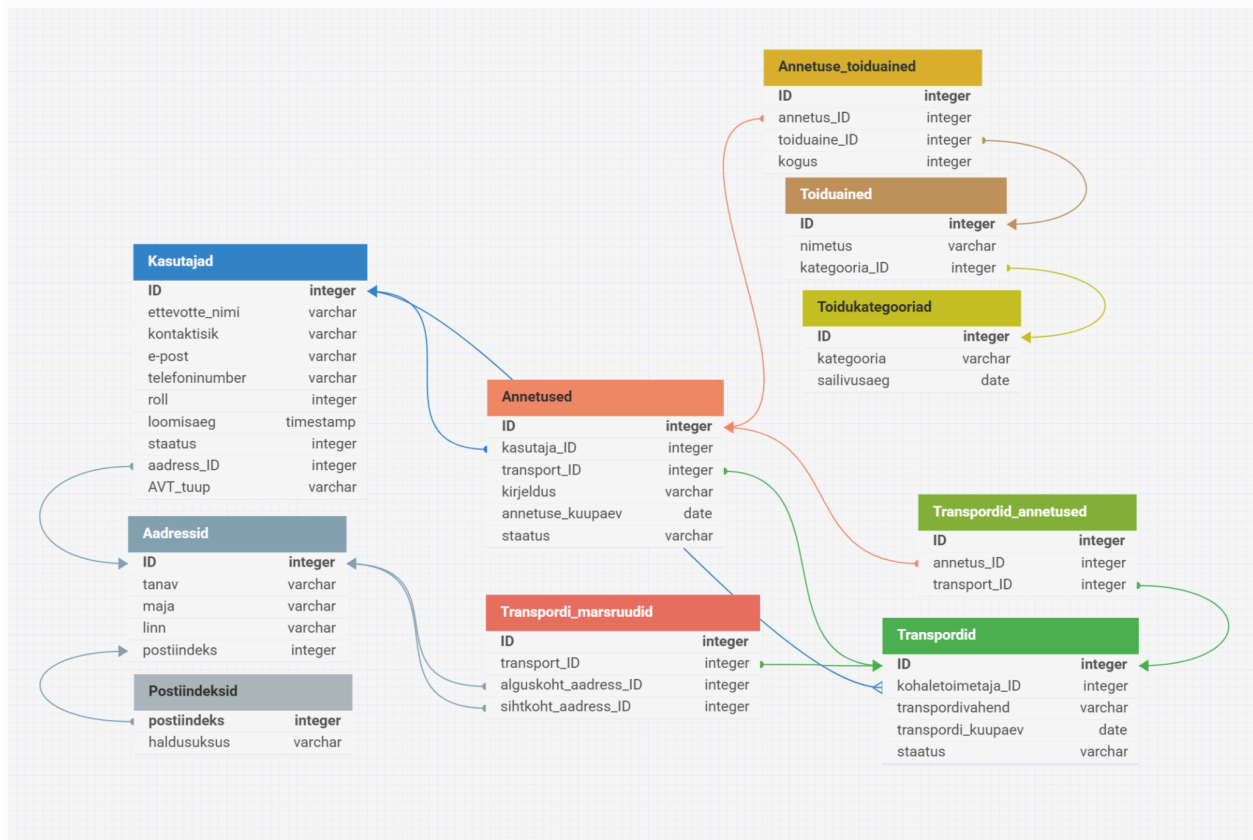
Dekompositsioon:

Toiduained (**nimetus**, kategooria)

Toidukategooriad (**kategooria**, säilivusaeg)

Uuendatud relatsiooniline andmebaasi mudel

(Kunstilikud võtmed on pärast 3. normaalkujule viimist tagasi lisatud)



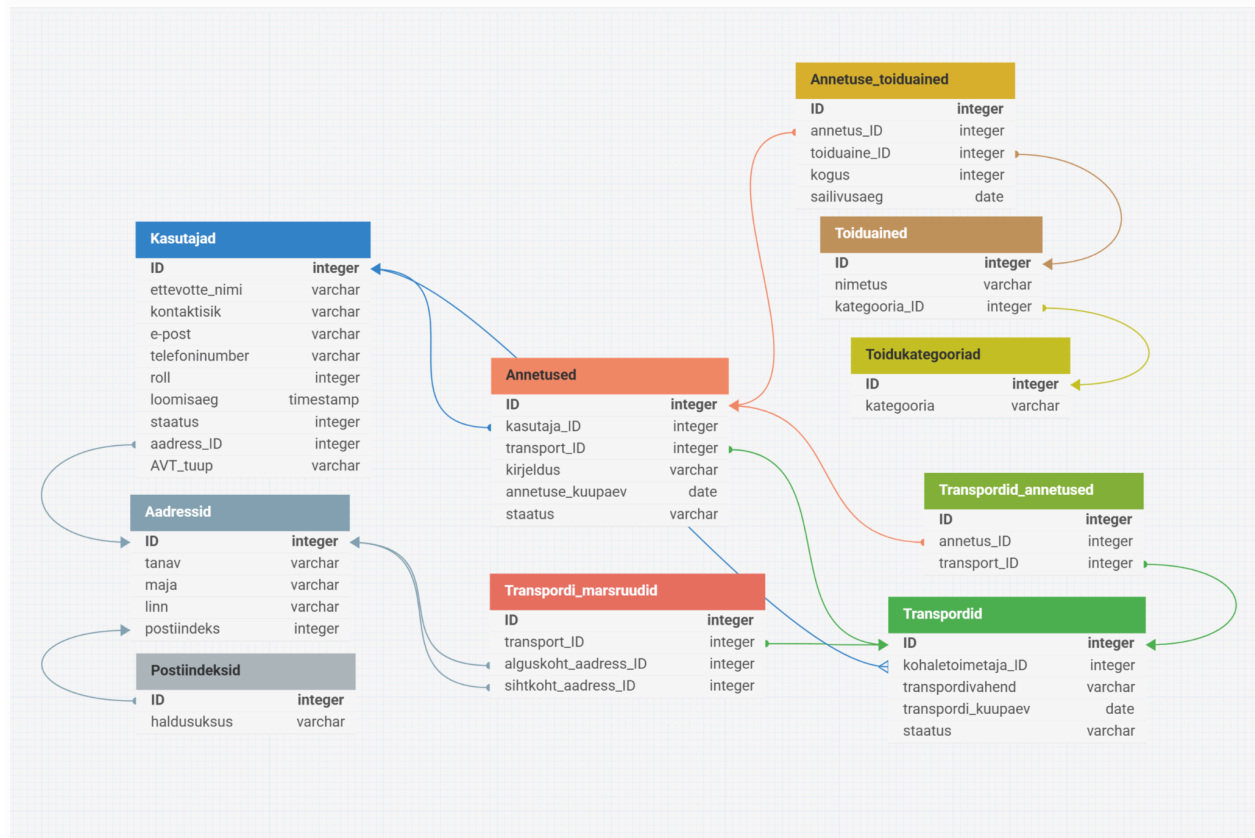
Etapp 3

Andmebaasi loomine

Muudatused

- Andmebaasi create lausetes sai parandatud "Aadressid" tabeli atribuudi "postiindeks" võtmestaatus, mis ei sobinud, kuna see ei olnud unikaalne.
- Pärast andmebaasi loomist tuli ilmsiks, et säilivusaja õige koht pole mitte "Toidukategooriad", vaid "Annetuse_toiduained" tabelis, sest sel juhul on võimalik määrata säilivusaeg igas annetuses eraldi märgitud toiduainele.
- Andmebaasis muudeti kõik atribuutides sisalduvad suurtähed, nagu "ID", väikeseks, et vältida nimekonflikte, mis võisid päringute koostamisel tekkida.
- Tabeli "Annetused" "transport_id" väljalt eemaldati NOT NULL seaduspärasus, kuna praktikas eksisteerib alati palju annetusi, mille pole veel vastuvõtjat leitud ega transporti määratud.

Lõplik andmebaasi mudel



Päringud

Andmebaasi peal saab teha kümneid erinevaid päringuid, kuid siin on välja toodud mõned huvitavamad näited, millest oleks kasu suurele osale andmebaasi kasutajatest..

Aegumisajad

Funktsioon “Aegumisajad” võtab argumentiks päevade arvu ning väljastab selle põhjal kõik toiduained, mis aeguvad nende päevade jooksul. Väljundis võivad korduda sama nimetusega toiduained, kuna sama nimega toiduainetel võib olla erinev säilivusaeg.

```
CREATE OR REPLACE FUNCTION Aegumisajad(paevad integer)
RETURNS TABLE (
  "annetuse id" bigint,
```

```

toiduaine varchar,
kogus bigint,
"säilivusaeg" date,
"päevi aegumiseni" integer
) AS $$
BEGIN
RETURN QUERY
SELECT
    a.id::bigint,
    t.nimetus,
    at.kogus,
    at.sailivusaeg,
    (at.sailivusaeg - CURRENT_DATE)
FROM annetused a
JOIN annetuse_toiduained at ON a.id = at.annetus_id
JOIN toiduained t ON at.toiduaine_id = t.id
WHERE (at.sailivusaeg - CURRENT_DATE) <= paevad;
END;
$$ LANGUAGE plpgsql;

```

Näidis 1:

`SELECT * FROM Aegumisajad(7);` -- Toiduained, mis aeguvad 7 päeva jooksul

Results 1 ×

`SELECT * FROM Aegumisajad(7)` | Enter a SQL expression to filter results (use Ctrl+Space)

	123 annetuse id	A-Z toiduaine	123 kogus	🕒 säilivusaeg	123 päevi aegumiseni	
1	10	leib	8	2025-05-10	5	
2	10	šokolaad	12	2025-05-08	3	
3	10	leib	6	2025-05-12	7	

- Antud näites esineb toiduainet "leib" kaks korda, sest kuigi nad kuuluvad samasse annetusse, on neil erinev säilivusaeg.

Näidis 2:

```
SELECT * FROM Aegumisajad(30); -- Toiduained, mis aeguvad 30 päeva jooksul
```

Results 1 ×

SELECT * FROM Aegumisajad(30) | Enter a SQL expression to filter results (use Ctrl+Space)

	123 annetuse id	A-Z toiduaine	123 kogus	⌚ säilivusaeg	123 päevi aegumiseni	
1	1	õun	10	2025-05-20	15	
2	1	piim	5	2025-05-18	13	
3	2	sealiha	15	2025-06-01	27	
4	2	leib	7	2025-05-25	20	
5	3	õun	12	2025-05-30	25	
6	10	leib	8	2025-05-10	5	
7	10	šokolaad	12	2025-05-08	3	
8	10	leib	6	2025-05-12	7	

- Kuni 30-päevase tähtajaga toiduaineid on tunduvalt rohkem. Väheste kirjete tõttu pole andmebaasis pole kahjuks aga tooteid, mis oleks üle tähtaja, mistõttu ei saa üle tähtaja läinud toiduaineid hetkel testida.

Hilinenud transpordid

Funktsioon "Hilinenud_transpordid" demonstreerib, kuidas oleks võimalik andmebaasi põhjal analüüsida logistilisi protsesse. Funktsioon tuvastab annetused, mis on olnud teel üle parameetris määratud aja, ning tagastab tabeli, kus on kõik sellised transpordid ilusti välja toodud.

```
CREATE OR REPLACE FUNCTION Hilinenud_transpordid(paevad integer)
RETURNS TABLE (
  "annetuse ID" bigint,
  kirjeldus varchar,
  "transpordi kuupäev" date,
  "senine transpordiaeg" integer → näitab, kui kaua on transpordiprotsess juba kestnud
) AS $$
BEGIN
  RETURN QUERY
  SELECT
    a.id::bigint AS "annetuse ID",
    a.kirjeldus,
```

```

t.transpordi_kuupaev AS "transpordi kuupäev",
CURRENT_DATE - t.transpordi_kuupaev AS "senine transpordiaeg"
FROM annetused a
JOIN transpordid_annetused ta ON a.id = ta.annetus_id
JOIN transpordid t ON ta.transport_id = t.id
WHERE lower(t.staatus) = 'teel'
AND (CURRENT_DATE - t.transpordi_kuupaev) > paevad;
END;
$$ LANGUAGE plpgsql;

```

Näidis:

SELECT * FROM hilinenud_transpordid(2);

Results 1 ×

SELECT * FROM hilinenud_transpordid(2) | Enter a SQL expression to filter results (use

	123 annetuse ID	A-Z kirjeldus	transpordi kuupäev	123 senine transpordiaeg
1	2	piimatooted	2025-05-02	3

Saadavalolevad annetused

Funktsioon "Saadavalolevad_annetused" demonstreerib vaadet andmetest, mis kuvatakse inimestele, kes uurivad andmebaasi mõne graafilise liidese, rakenduse või platvormi kaudu, et saada ülevaade, milliseid annetusi parasjagu pakutakse. Funktsioon tagastab tabeli, kus on annetused vormistatud üksteise järel olevate plokkidena, mida on võimalik eristada esimeses tulbas olevate annetuse numbrite ehk ID-de järgi. Ploki alguses on alati info postitaja kohta (tavaliselt lisatakse firmanimi, kuid kui seda ei ole või tegemist on eraisikuga, siis lisatakse kontaktisiku nimi) ja samuti on kohe näha, kui kaua aega tagasi mingi 'kuulutus' lisati (kuvatakse ainult annetusi, millel pole veel transporti ehk mida keegi pole veel nõus vastu võtma). Kirjeldavale reale järgneb annetuses sisalduvate toiduainete loetelu, kus on välja toodud toiduaine nimetus, selle kogus ja eeldatav säilivusaeg.

```

CREATE OR REPLACE FUNCTION Saadavalolevad_annetused()
RETURNS TABLE (
    "annetuse number" bigint,
    kirjeldus text
) AS $$
BEGIN
    RETURN QUERY
    SELECT *
    FROM (
        -- Annetuse üldandmed
        SELECT
            a.id::bigint AS "annetuse number",
            COALESCE(k.ettevotte_nimi, k.kontaktisik) || ', ' ||
            (CURRENT_DATE - a.annetuse_kuupaev) || ' päeva tagasi' AS kirjeldus
        FROM annetused a
        JOIN kasutajad k ON a.kasutaja_id = k.id
        LEFT JOIN transpordid_annetused ta ON a.id = ta.annetus_id
        WHERE ta.transport_id IS NULL
        UNION ALL
        -- Toiduainete read
        SELECT
            a.id::bigint AS "annetuse number",
            t.nimetus || ' (' || at.kogus || ' tk), ' || TO_CHAR(at.sailivusaeg, 'DD-MM-YYYY') AS kirjeldus
        FROM annetused a
        JOIN annetuse_toiduained at ON a.id = at.annetus_id
        JOIN toiduained t ON at.toiduaine_id = t.id
        LEFT JOIN transpordid_annetused ta ON a.id = ta.annetus_id
        WHERE ta.transport_id IS NULL
    ) AS combined
    ORDER BY "annetuse number", kirjeldus;
END;
$$ LANGUAGE plpgsql;

```

Näidisväljund:

`SELECT * FROM Saadavalolevad_annetused();`

Results 1 ×

`SELECT * FROM Saadavalolevad_annetused();` Enter a SQL exp

	123 annetuse number	A-Z kirjeldus
1	10	Laura Leht, 0 päeva tagasi
2	10	leib (6 tk), 12-05-2025
3	10	leib (8 tk), 10-05-2025
4	10	šokolaad (12 tk), 08-05-2025

- Nagu näha, siis hetkel on vastuvõtjata ainult üks annetus.

Kuine statistika

Vaade “Kuine_annetuste_statistika” demonstreerib andmebaasi mõningaid statistilise analüüsi võimalusi, andes kuude kaupa ülevaate kõikide toidukategooriate üldisest statistikast. Vaates on näha, mis on iga kategooria annetuste arv, toiduainete koguarv ja vastava kategooria aktiivsete annetajate arv.

```
CREATE OR REPLACE VIEW Kuine_annetuste_statistika AS
SELECT
  DATE_TRUNC('month', a.annetuse_kuupaev) AS kuu,
  k.kategooria,
  COUNT(DISTINCT a.id) AS "annetuste arv",
  SUM(at.kogus) AS "toiduainete koguarv",
  COUNT(DISTINCT a.kasutaja_id) AS "aktiivseid annetajaid"
FROM annetused a
JOIN annetuse_toiduained at ON a.id = at.annetus_id
JOIN toiduained t ON at.toiduaine_id = t.id
JOIN toidukategooriad k ON t.kategooria_id = k.id
GROUP BY kuu, k.kategooria
ORDER BY kuu DESC, k.kategooria;
```

Vaade:

	kuu	A-Z kategooria	123 annetuste arv	123 toiduainete koguarv	123 aktiivseid annetajaid
1	2025-05-01 00:00:00.000 +0300	liha	1	6	1
2	2025-05-01 00:00:00.000 +0300	maiustused	1	12	1
3	2025-05-01 00:00:00.000 +0300	teraviljatooted	1	8	1
4	2025-04-01 00:00:00.000 +0300	liha	1	15	1
5	2025-04-01 00:00:00.000 +0300	piimatooted	1	5	1
6	2025-04-01 00:00:00.000 +0300	puu- ja köögiviljad	2	22	2
7	2025-04-01 00:00:00.000 +0300	teraviljatooted	1	7	1

Tegevused

Jällegi – andmebaasil on palju erinevaid osalisi ja detaile, mistõttu saaks koostada lõputul arvul erinevaid huvitavaid protseduure, kuid hetkel on piirdume ainult kahe näidisega.

Uue annetuse lisamine

Andmebaasi uue annetuse lisamiseks on protseduur "Lisa_annetus", mis võimaldab lisada ühte annetusse mitu erinevat toiduainet.

```
CREATE OR REPLACE PROCEDURE Lisa_annetus(
  IN p_kasutaja_id bigint, → annetaja ID
  IN p_kirjeldus varchar, → annetuse kirjeldus (nt "külmtoidud")
  IN p_kuupaev date, → annetuse kuupäev
  IN p_staatus varchar, → staatus (nt "ootel" või "teel")
  IN p_toiduaine_id bigint[], → toiduainete ID-d järjendina, et oleks lihtsam sisestada
  IN p_kogused bigint[], → vastavad kogused
  IN p_sailivusajad date[] → vastavad säilivusajad
)
LANGUAGE plpgsql
AS $$
DECLARE
  annetus_id bigint;
  i integer;
BEGIN
```

```

INSERT INTO annetused(kasutaja_id, transport_id, kirjeldus, annetuse_kuupaev, staatus)
VALUES (p_kasutaja_id, NULL, p_kirjeldus, p_kuupaev, p_staatus)
RETURNING id INTO annetus_id;
FOR i IN 1..array_length(p_toiduaine_id, 1) LOOP
    INSERT INTO annetuse_toiduained(annetus_id, toiduaine_id, kogus, sailivusaeg)
    VALUES (annetus_id, p_toiduaine_id[i], p_kogused[i], p_sailivusajad[i]);
END LOOP;
END;
$$;

```

Viimased kolm parameetrit on järjendid, kuna iga annetus võib sisaldada rohkem kui ühte toiduainet. Kõik need massiivid peavad olema sama pikkusega, sest iga indeks tähistab ühte toiduainet.

Näide triggeri aktiveerumisest ehk sellest, mis juhtub siis, kui mingit toiduainet ei eksisteeri:

```

CALL lisa_annetus(
  3, -- restorani kasutaja_ID
  'Restorani ülejäägid: supid ja salatid',
  CURRENT_DATE,
  'aktiivne',
  ARRAY[4, 5, 6], -- toiduaine_id: nt 4=Supp, 5=Salat, 6=Leib
  ARRAY[8, 12, 6], -- kogused vastavalt
  ARRAY['2025-05-10', '2025-05-08', '2025-05-12']::date[] -- säilivusajad, millel on kuupäeva vorming
);

```

SQL Error [P0001]: ERROR: Toiduaine ID 6 ei eksisteeri toiduainete tabelis! Lisage puuduvad toiduained see enne annetuse sisestamist!

Where: PL/pgSQL function kas_toiduaine_eksisteerib() line 8 at RAISE

SQL statement "INSERT INTO annetuse_toiduained(annetus_id, toiduaine_id, kogus, sailivusaeg) VALUES (annetus_id, p_toiduaine_id[i], p_kogused[i], p_sailivusajad[i])"

PL/pgSQL function lisa_annetus(bigint,character varying,date,character varying,bigint[],bigint[],date[]) line 11 at SQL statement

Näide korrektsest tulemusest (lisatud 6. rida):

	123 id	123 kasutaja_id	123 transport_id	A-Z kirjeldus	annetuse_kuupaev	A-Z staatus
1	1	1	1	puu- ja juunviljad	2025-04-28	uus
2	2	2	2	päimatooted	2025-04-29	töös
3	3	3	3	liha	2025-04-30	lõpetatud
4	4	4	4	leivad	2025-05-01	uus
5	5	5	5	maiustused	2025-05-02	tühistatud
6	10	3	[NULL]	Restorani ülejäägid: supid ja salatid	2025-05-05	aktiivne

Triggeri konfiguratsioon:

```
CREATE OR REPLACE FUNCTION Kas_toiduaine_eksisteerib() RETURNS TRIGGER AS $$
DECLARE
    olemas integer;
BEGIN
    SELECT COUNT(*) INTO olemas FROM toiduained WHERE id = NEW.toiduaine_id;
    IF olemas = 0 THEN
        RAISE EXCEPTION 'Toiduaine ID % ei eksisteeri toiduainete tabelis! Lisage puuduvad toiduained
see enne annetuse sisestamist!', NEW.toiduaine_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER Toiduaine_eksisteerib
BEFORE INSERT ON annetuse_toiduained
FOR EACH ROW
EXECUTE FUNCTION Kas_toiduaine_eksisteerib();
```

Annetuse staatuse uuendamine

Annetuste uuendamiseks on loodud protseduur “Uuenda_annotuse_staatus”, mis võtab argumentideks annetuse ID ja uue sõnalise staatuse väärtuse, olgu see ükskõik milline staatuse variant, mis on andmebaasis ette nähtud.

```
CREATE OR REPLACE PROCEDURE Uuenda_annotuse_staatus(
    p_annotus_id bigint,
    p_uus_staatus varchar
)
LANGUAGE plpgsql
AS $$
BEGIN
```

```
UPDATE annetused
SET sataatus = p_uus_staatus
WHERE id = p_annotus_id;
END;
$$;
```

Näidis (viimase rea staatus on muudetud):

```
--SELECT * FROM annetused;

CALL Uuenda_annotuse_staatus(1, 'kohale toimetatud');
```

annetused 1 ×						
SELECT * FROM annetused Enter a SQL expression to filter results (use Ctrl+Space)						
	123 id	123 kasutaja_id	123 transport_id	A-Z kirjeldus	annetuse_kuupaev	A-Z staatus
1	2	2	2	piimatooted	2025-04-29	töös
2	3	3	3	liha	2025-04-30	lõpetatud
3	4	4	4	leivad	2025-05-01	uus
4	5	5	5	maiustused	2025-05-02	tühistatud
5	10	3	[NULL]	Restorani ülejäägid: supid ja salatid	2025-05-05	aktiivne
6	1	1	1	puu- ja juurviljad	2025-04-28	kohale toimetatud

Joosep Tiger Tilgar